SEEN - PE

Synthèse de la mission SunTracker

GRANDE ÉCOLE D'INGÉNIEURS

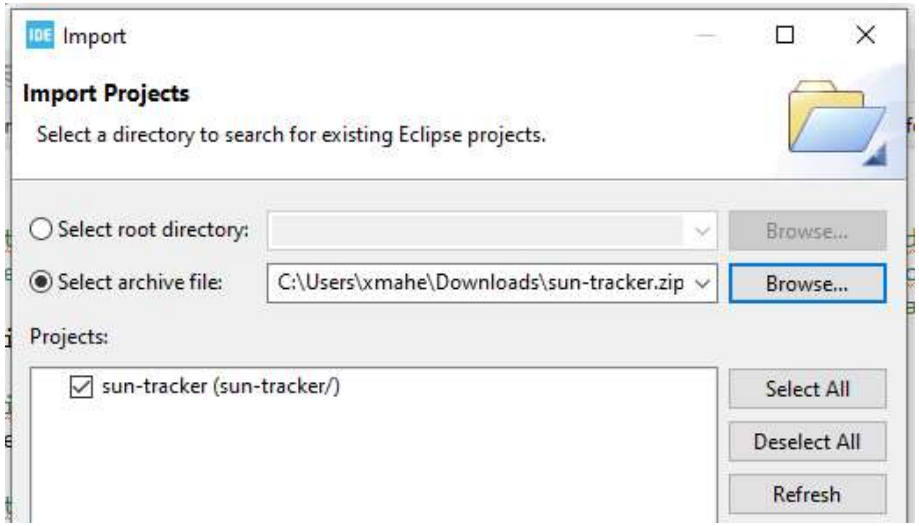
Objectif de la mission

- **Objectifs**
  1. CubeIDE - Les bonnes pratiques
  2. Mise en œuvre du débogueur
  3. Tableaux et chaînes de caractères
  4. Ecriture d'un module logiciel : *fichiers .h et .c*
  5. Utilisation d'un Timer: *génération d'un signal rectangulaire*
  6. Structures de données et pointeurs
  7. Temps d'exécution d'une fonction
  8. Utilisation du périphérique ADC
  9. Réalisation d'une machine à états

CubeIDE - Bonnes pratiques

- Importation d'un projet:
  - Télécharger le projet "sun-tracker.zip" à partir du Moodle
  - Laisser le fichier "sun-tracker.zip" dans le dossier de téléchargement
  - Importer le fichier dans CubeIDE :

    File / Import / General / Existing project into workspace



Mise en œuvre du débogueur

Apprendre "Le C embarqué par l'exemple.pdf"
(cf. Moodle)

- Appel de la fonction ***useless_function()***
- **Mise en œuvre du débogueur**

(x)= Variables | Breakpoints | Expressions | Modules | Registers

Expression	Type	Value
(x)= a	volatile uint8_t	0xca (Hex)
(x)= b	volatile int	1
(x)= c	volatile uint8_t	0 (Binary)
(x)= d	volatile uint32_t	0xcafedeca (Hex)
Add new expression		

- **Utilisation des boutons de défilement en pas à pas :**
  - **Step Into (F5)**

    Exécute la ligne de code actuelle et pointe à la ligne de code suivante.

    Chaque fonction appelée est exécutée ligne à ligne.

  - **Step Over (F6)**

    Exécute la ligne de code actuelle et pointe à la ligne de code suivante.

    Chaque fonction appelée est exécutée intégralement en une seule fois.

Tableaux et chaines de caractères

```c
void SECRETARY_process_main(void)
{
    //Définition d'un tableau de BUFFER_SIZE caractères, de type 'static char'
    static char buffer[BUFFER_SIZE];

    //Définition d'un entier de type 'static uint8_t', nommé index (initialisé à 0)
    static uint8_t index = 0;

    //Définition d'une variable de type char, nommée c
    char c;

    //Tant que (Des données sont disponibles sur l'UART2, cf fonction UART_data_ready)
    while (UART_data_ready(UART2_ID))
    {
        c = UART_getc(UART2_ID);

        //ranger c dans la case index du tableau buffer
        buffer[index] = c;

        if (c == '\n')
        {
            buffer[index] = '\0';
            //echo_command(buffer);
            find_command(buffer);
            index = 0;
        }

        else if (index < (BUFFER_SIZE - 1))
        {
            index++;
        }
    }
}
```

Apprendre "Le C embarqué par l'exemple.pdf"
(cf. Moodle)

!!! IMPORTANT !!!

Soigner la présentation
Indenter son code
Commenter son code

Indenter son code
Commenter son code

... etc.

Ecriture d'un module logiciel

Ecriture du Header

```
 * servo.h

#ifndef BSP_SERVO_SERVO_H_
#define BSP_SERVO_SERVO_H_

//inclusions nécessaires à l'interprétation des prototypes publics ci-dessous
#include "config.h"
#include "stm32f1_timer.h"
#include "stm32f1_gpio.h"
#include "macro_types.h"

//Prototypes des fonctions publiques
void SERVO_init(void);
void SERVO_set_position(uint16_t position);
void SERVO_process_test(void);
uint16_t SERVO_get_position(void);

#endif /* BSP_SERVO_SERVO_H_ */
```

Ecriture d'un module logiciel

Définition des fonctions dans le fichier .c

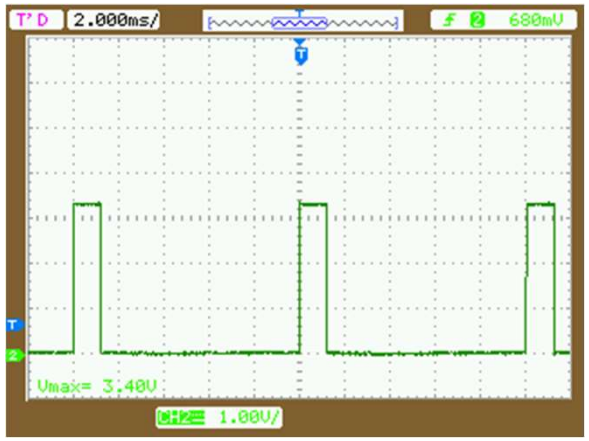
```c
#define PERIOD_TIMER 10

void SERVO_init(void)
{
    //initialisation et lancement du timer1 à une période de 10 ms
    TIMER_run_us(TIMER1_ID, PERIOD_TIMER*1000, FALSE);

    //10000us = 10ms //activation du signal PWM sur le canal 1 du timer 1 (broche PA8)
    TIMER_enable_PWM(TIMER1_ID, TIM_CHANNEL_1, 150, FALSE, FALSE);

    //rapport cyclique reglé pour une position servo de 20%
    SERVO_set_position(20);
}

//position est exprimée de 0 à 100%.
void SERVO_set_position(uint16_t position)
{
    current_position = position;

    if(position > 100)
        position = 100; //écretage si l'utilisateur demande plus de 100%

    //TODO : mise à jour du rapport cyclique.
    //duty doit être exprimé ici de 100 à 200 (sur 1000)
    //Pour un rapport cyclique de 10% à 20%,
    //c'est-à-dire une durée de pulse de 1 à 2ms dans la période de 10ms
    //Donc on additionne 100 à position.
    TIMER_set_duty(TIMER1_ID, TIM_CHANNEL_1,(uint16_t)(position+100));
}
```

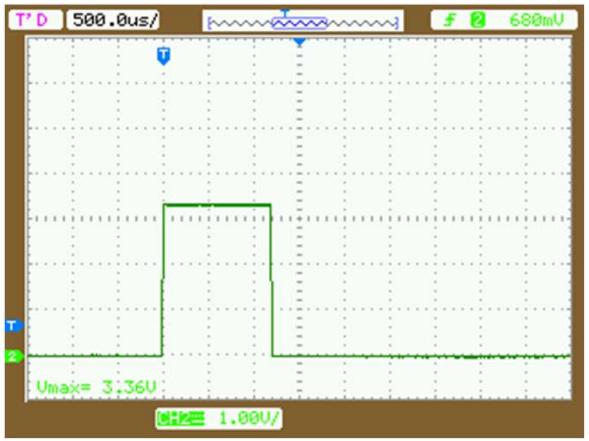
Utilisation d'un Timer

Pour générer un signal rectangulaire sur la broche PA8, on configure le CH1 du Timer 1

Signal obtenu en PA8 pour une commande de 20% du servomoteur
(Th = 1,2ms = 1200µs)

```
SERVO_set_position(20);
```





Structures de données et pointeurs

La définition du type de structure **SolTrackLocation_t** est contenue dans "SolTrack.h"

```
/// @brief Location to compute the Sun's position for
typedef struct {
  double longitude, latitude;
  double sinLat, cosLat;
  double pressure, temperature;
}SolTrackLocation_t;
```

La structure **loc** de type **SolTrackLocation_t** est déclarée **dans "sun_tracker.c"**

```
static SolTrackLocation_t loc;
```

La fonction **SUN_TRACKER_set_new_pos** est définie **dans "sun_tracker.c"**

```
void SUN_TRACKER_set_new_pos(double longitude, double latitude)
{
    loc.longitude = longitude;
    loc.latitude = latitude;
}
```

La structure **loc** est **locale au fichier**, d'où le point !!!

Structures de données et pointeurs

```c
/// @brief Date and time to compute the
typedef struct {
  int year, month, day, hour, minute;
  double second;
}SolTrackTime_t;

void SUN_TRACKER_process_main(void)
{
    SECRETARY_process_main();
    if(flag_1s)
    {
        HAL_GPIO_TogglePin(LED_GREEN_GPIO, LED_GREEN_PIN);
        flag_1s = FALSE;
        SolTrackTime_t time;
        //données de test temporaires.
        time.year =     (int32_t)(20) + 2000;   //2020
        time.month =    (int32_t)(4);           //avril
        time.day =      (int32_t)(1);           //1er
        time.hour =     (int32_t)(8);
        time.minute =   (int32_t)(30);
        time.second =   (double)(40.4);


        //variable qui serviront à receuillir les résultats des calculs
        SolTrackPosition_t pos;
        SolTrackRiseSet_t riseSet;

        //calculs
        SUN_TRACKER_compute_sun_position(&time, &pos, &riseSet);
    }
}
```

La structure **time** est déclarée dans la fonction "SUN_TRACKER_process_main()"
Elle n'est accessible que dans la fonction "SUN_TRACKER_process_main()"

On indique **l'adresse (&)** de la structure **time** à la fonction "SUN_TRACKER_compute_sun_position" lors de son appel.

GRANDE ÉCOLE D'INGÉNIEURS

Structures de données et pointeurs

La fonction "SUN_TRACKER_compute_sun_position" utilise **l'adresse contenue dans le pointeur temps** pour accéder aux champs de la structure **time**

```
static void SUN_TRACKER_compute_sun_position(SolTracktemps_t * temps, SolTrackPosition_t * pos, S
{
    // Compute rise and set tempss:
    SolTrack_RiseSet(temps, &loc, pos, riseSet, 0.0, USE_DEGREES, USE_NORTH_EQUALS_ZERO);

    // Compute positions:
    SolTrack(temps, &loc, pos, USE_DEGREES, USE_NORTH_EQUALS_ZERO, COMPUTE_REFRACTION_EQUATORIAL,

    #if ENABLE_DISPLAY
    // Write data to screen:
    printf("Date\t\t\t\t:\t%2d %2d %4d\n", temps->month, temps->day, temps->year);
    printf("Heure\t\t\t\t:\t%2d %2d %2.1lf\n", (int)temps->hour, (int)temps->minute, temps->secor
    printf("Jour julien\t\t\t:\t%7.2lf\n\n", pos->julianDay);

    printf("Heure levé\t\t\t:\t%2.4lf \tazimuth:   %3.2lf\n",  riseSet->risetemps,  riseSet->riseA
```

temps est un pointeur vers la structure **time**, d'où la flèche !!!

Temps d'exécution d'une fonction

On cherche à connaître le temps d'exécution de la fonction **SUN_TRACKER_compute_sun_position**

```
//calculs
HAL_GPIO_WritePin(LED_GREEN_GPIO, LED_GREEN_PIN,GPIO_PIN_SET);
SUN_TRACKER_compute_sun_position(&time, &pos, &riseSet);
HAL_GPIO_WritePin(LED_GREEN_GPIO, LED_GREEN_PIN,GPIO_PIN_RESET);
```

Le temps d'exécution de la fonction **SUN_TRACKER_compute_sun_position** diffère **selon que l'on active ou pas** l'écriture des données dans Docklight

```
#if ENABLE_DISPLAY
// Write data to screen:
printf("Date\t\t\t\t:\t%2d %2d %4d\n", temps->month, temps->day, temps
printf("Heure\t\t\t\t:\t%2d %2d %2.1lf\n", (int)temps->hour, (int)temp
printf("Jour julien\t\t\t:\t%7.2lf\n\n", pos->julianDay);

printf("Heure levé\t\t\t:\t%2.4lf,\tazimuth:   %3.2lf\n", riseSet->ris
printf("Heure zénith\t\t\t:\t%2.4lf,\taltitude:  %3.2lf\n", riseSet->t
printf("Heure couché\t\t\t:\t%2.4lf,\tazimuth:   %3.2lf\n\n", riseSet-

printf("Ecliptic longitude\t\t:\t%2.2lf\n", pos->longitude);
printf("Ascension droite et déclinaison :\t%2.2lf %2.2lf\n", pos->righ
printf("altitude non corrigée\t\t:\t%2.2lf\n\n", pos->altitude);
printf("azimuth et altitude corrigées\t:\t%3.2lf %2.2lf\n\n", pos->azi
#endif
```

```
Date          :    1  4 2020
Heure         :    8:30:40.4
Jour julien   :    2458940.85

Heure levé    :   5.6722,     azimuth  :  82.08
Heure zénith  :   12.0988,    altitude :  47.34
Heure couché  :   18.5407,    azimuth  :  278.23

Ecliptique longitude :                12.07°
ascension droite et déclinaison :     11.10°        4.77°
altitude non corrigée :               27.31°

azimuth et altitude corrigées :       115.13°       27.34°
```

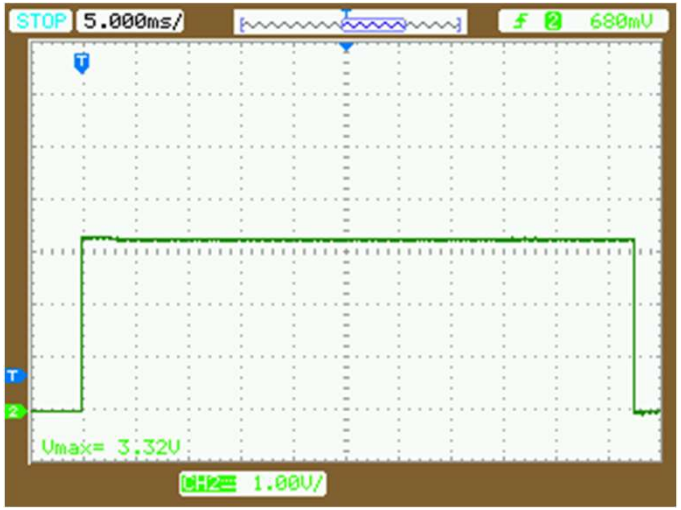
Temps d'exécution d'une fonction

Sans écriture des données dans Docklight

T = 18ms

Avec écriture des données dans Docklight

T = 55ms



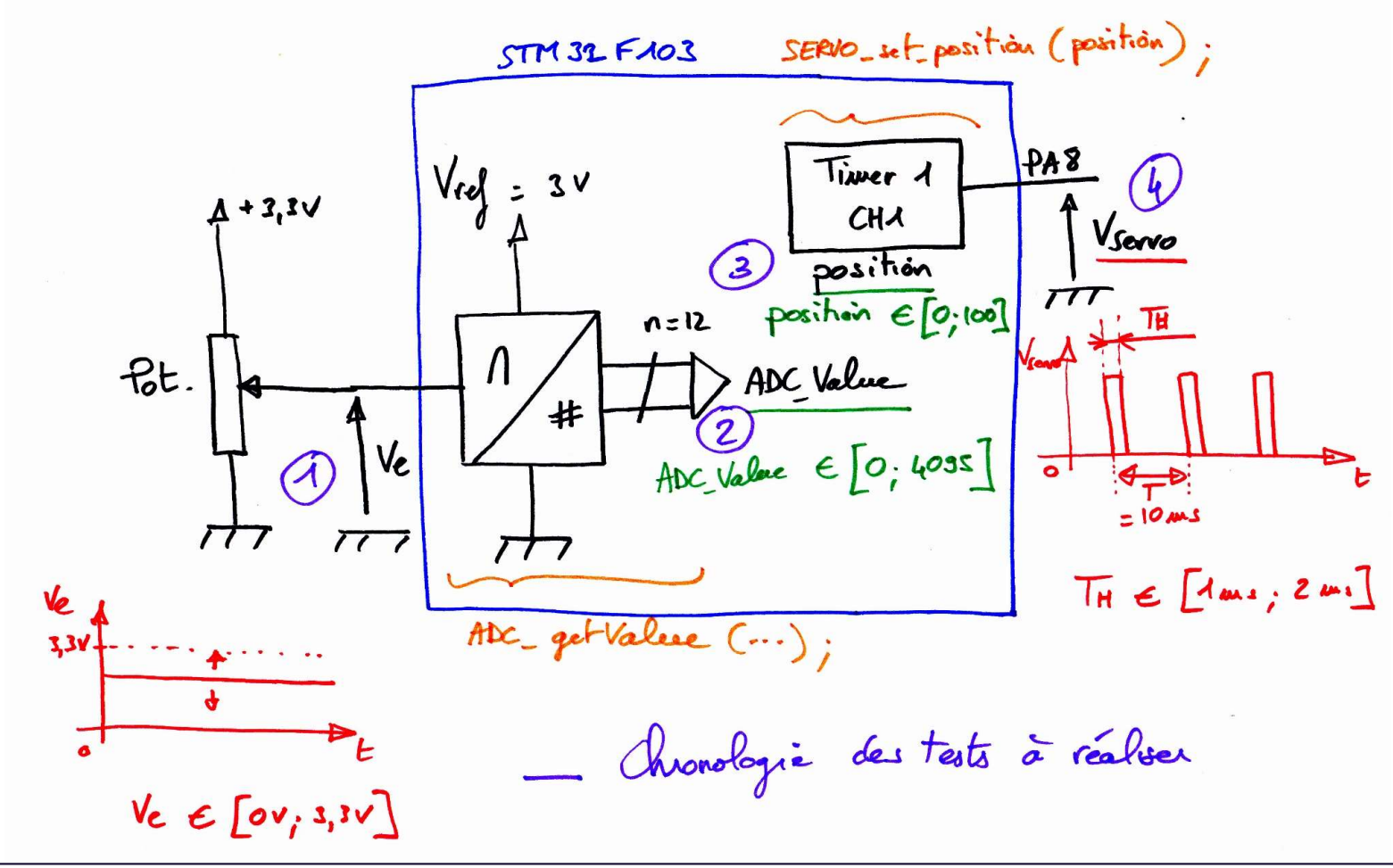
IMPORTANT

Information à prendre en compte lorsque la fonction est appelée dans un processus dont le temps d'exécution est limité.

Exemple :

Dans ce cas, si temps du processus < 55ms, cela fera planter votre programme !!!

Utilisation du périphérique ADC



Utilisation du périphérique ADC

Configuration dans le fichier « config.h »

```
#define USE_BSP_TIMER       1
#define USE_BSP_EXTIT       1

#define USE_RTC             1

#define USE_ADC             1
    /* Configuration pour activer les entrÃ©es analogiques souhaitÃ©es */
    #define USE_IN1     1 //Broche correspondante: PA0
```

Initialisation du périphérique ADC

```
void SUN_TRACKER_init(void)
{
    BSP_ADC_init();
    BSP_RTC_init();
    BSP_RTC_set_time_acceleration(TIME_ACCELERATION);
```

Lancement d'une acquisition

```
    //variable qui serviront Ã  receuillir les rÃ©sultats des calculs
    SolTrackPosition_t pos;
    SolTrackRiseSet_t riseSet;

    //calculs
    position = (uint16_t) ((BSP_ADC_getValue(ADC_1)*100)/4095);
    SERVO_set_position(position);
    SUN_TRACKER_compute_sun_position(&time, &pos, &riseSet);
```

Réalisation d'une machine à état

Suivre les indications en commentaire

```c
static void SUN_TRACKER_mode_management(void)
{
	//Definition d'une énumération mode_e définissant 3 modes :
	//MODE_SUN_TRACKER,MODE_MANUAL, MODE_FAST_ROUND_RETURNS

	//créez la variable 'mode', static de type mode_e, initialisé à MODE_SUN_TRACKER
	//créez un entier 'position', non signé sur 16 bits

	if(!t_read_button)		//le chronomètre est épuisé
	{
		t_read_button = 10;//on recharge le chronomètre pour 10ms

		//ajoutez ici un switch sur la variable mode
		//ce switch reprend l'ensemble des valeurs de l'énumération
		/*
		dans le MODE_MANUAL :
			//TODO acquérir la valeur du potentiomètre
			//position = ...;
			Si(évènement bouton)
			{
				//passer au MODE_SUN_TRACKER
			}

		dans le MODE_SUN_TRACKER :
			position = sun_tracker_position;
			Si(évènement bouton)
			{
				//passer au MODE_FAST_ROUND_RETURNS
			}
		dans le mode MODE_FAST_ROUND_RETURNS :
			imaginez un algo pour faire évoluer la position de 1 à chaque appel
			avec changement de sens quand on arrive aux extrêmes (0 ou 100)
			Si(évènement bouton)
			{
				//passer au MODE_MANUAL
			}
		*/
		//Pilotage du servomoteur avec l'argument 'position'
	}
}
```

Apprendre "Le C embarqué par l'exemple.pdf"
(cf. Moodle)